Udono Shuets, *Japanese Car Accident*. Tokyo, Japan. 2007.

Synkronisering:
Racetillstånd och den kritiska sektionen

Jimmy Åhlander

2023-11-14

Mittuniversitetet
MID SWEDEN UNIVERSITY

Agenda

- Racetillstånd (race condition).
- Kritisk sektion (critical section).
- Lösningar till det kritiska sektionsproblemet (*critical-section problem*):
  - Stänga av interrupts.
  - Strikt alternering.
  - Peterson’s solution.
  - Mutex-lås.
  - Semaforer.
- Readers-writers problem.
- Exempel i Python.

Racetillstånd (race condition)

- Uppstår när två eller flera processer *samtidigt* försöker modifiera och läsa en gemensam resurs, utan synkronisering, där utfallet beror på ordningen processerna får exekvera.

- En resurs kan exempelvis vara delad data, så som en variabel.

Ett exempel

```
x = 0
```

P_0	P_1
`x = x + 2`	
	`x = x + 5`

tid

```
print(x) // x == 7  // x == 5
```

Ett exempel (maskinkod)

```
x = 0
```

P_0	P_1
`LOAD X`	
`ADD 2,X`	
	`LOAD X`
`STORE X`	
	`ADD 5,X`
	`STORE X`

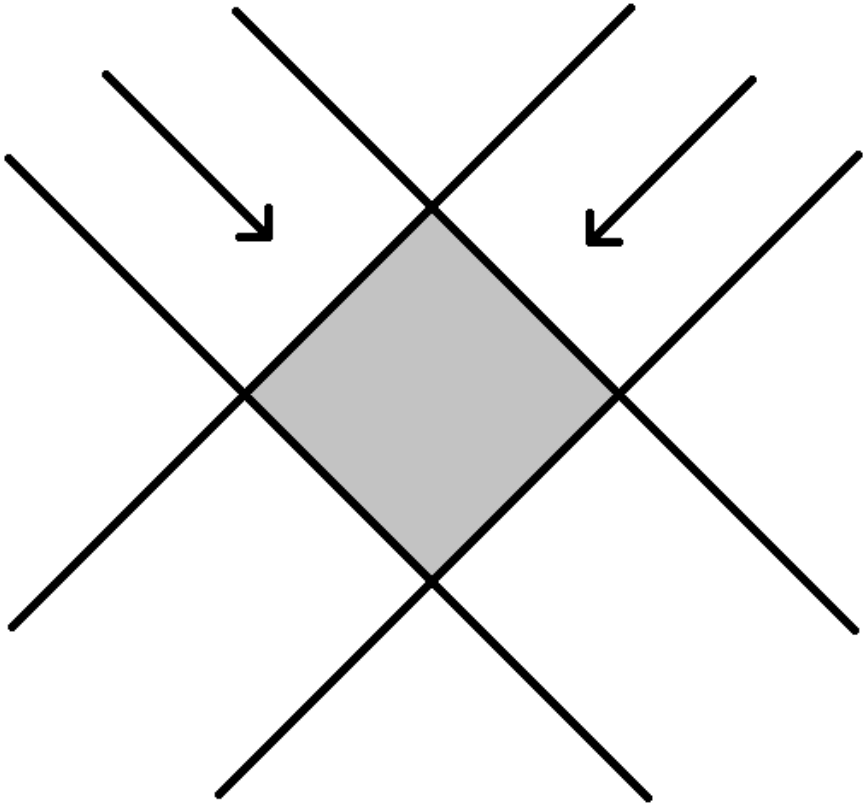
`x = x + 2`

tid

```
print(x) // x == 5
```

Kritisk sektion (critical section)

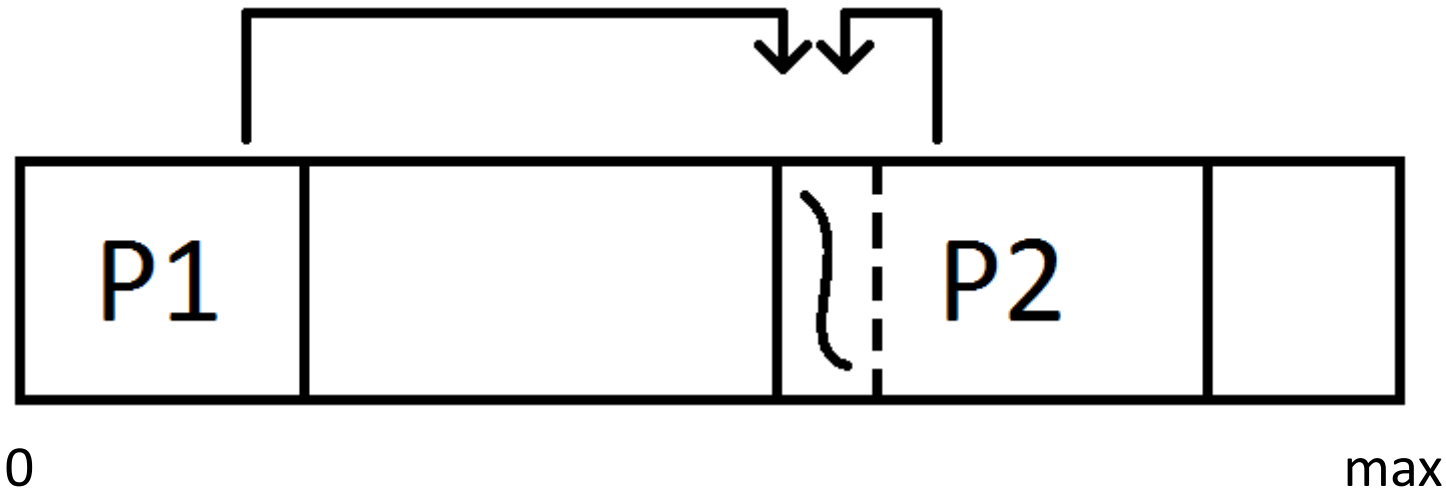
- Koden i ett program som har tillgång till delade resurser kallas för en kritisk sektion.



- Koden måste *synkroniseras* för att lösa det kritiska sektionsproblemet (the *critical section problem*).

Kritisk sektion (critical section)

Delat minne



The critical section problem
Krav på lösningar

1. **Mutual exclusion**

   Två processer får aldrig befinna sig i en viss kritisk sektion samtidigt.*

2. **Progress**

   Om ingen process exekverar i en viss kritisk sektion och det existerar någon process som vill exekvera i den kritiska sektionen så får inte beslutet fördröjas oändligt länge.

3. **Bounded waiting**

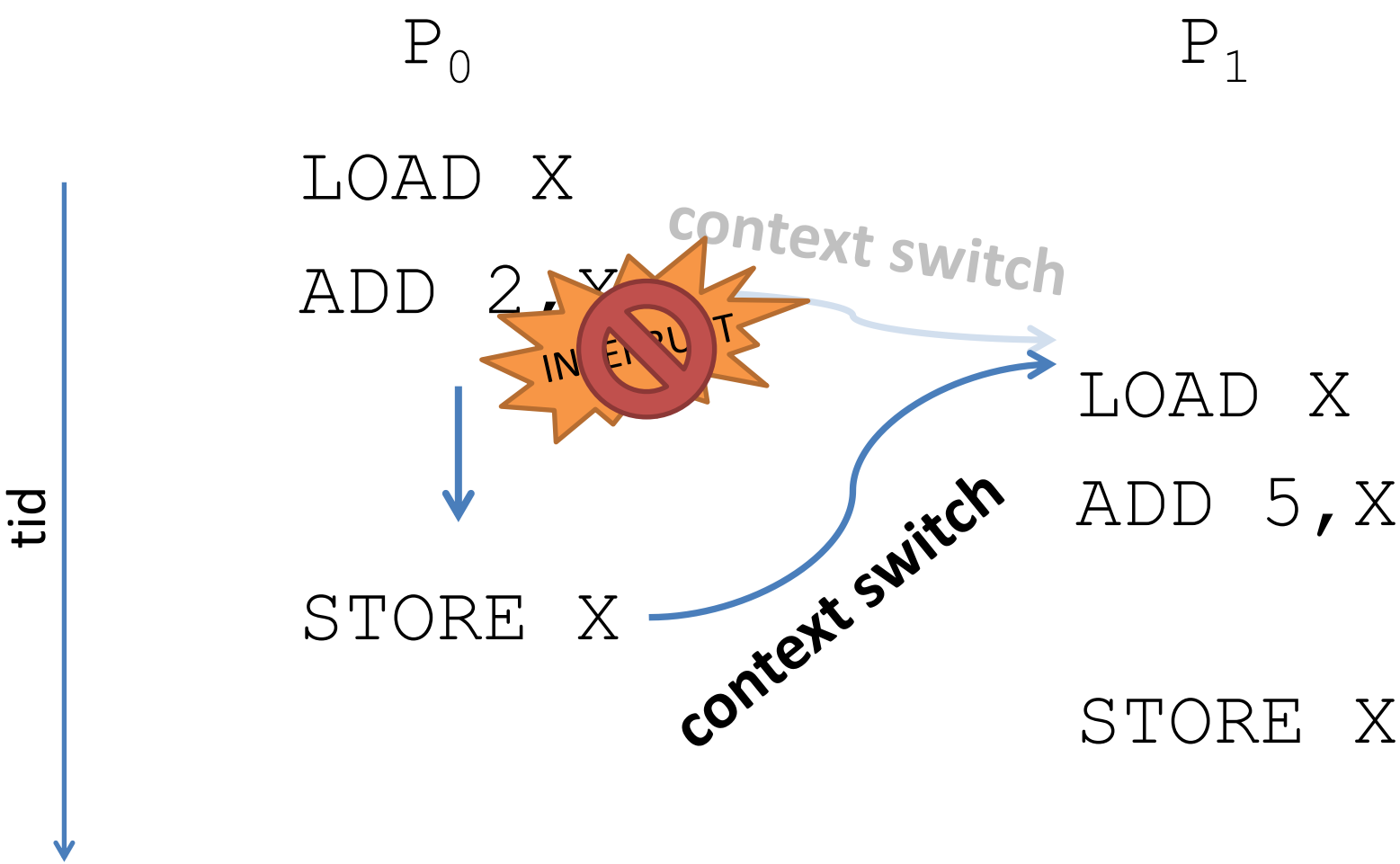
   Det måste finnas en gräns för hur många gånger andra processer får gå in i sina kritiska sektioner innan en viss process får gå in i sin.

* Om processerna endast läser är det ok.

Lösningar som kan implementeras helt, eller nästan helt, genom mjukvara

- Stänga av interrupts.
- Strikt alternering.
- Peterson’s solution.
- Icke-fungerande lås.

En enkel lösning: stäng av interrupts



En enkel lösning: stäng av interrupts

+ Simpel.

- Stoppar context switching helt och hållet.
  - Ger mycket makt till användarprocesser.
  - Vi tappar time-sharing, multitasking.

Generell struktur
för kommande exempel

```
// initialt tillstånd
```

P_0

```
while (True):
   // entry section
   // critical section
   // exit section

   // non-critical section
```

P_1

```
while (True):
   // entry section
   // critical section
   // exit section

   // non-critical section
```

Strikt alternering (strict alternation)

```
turn = 0
```

P_0

```
while (True):
   while (turn != 0):
        pass
   // critical section
   turn = 1

   // non-critical section
```

P_1

```
while (True):
   while (turn != 1):
        pass
   // critical section
   turn = 0

   // non-critical section
```

Strikt alternering (strict alternation)

+ Simpel.

- Slösar mycket tid på *busy-waiting* (spinning).
- Processer <u>måste</u> alternera att gå in i sin kritiska sektion.
  - Uppfyller därför inte kravet för progress.

Peterson's solution

```
turn = 0
interested[0] = False
interested[1] = False
```

P_0

```
while (True):
   interested[0] = True
   turn = 1
   while (interested[1] and
   turn == 1):
        pass
   // critical section
   interested[0] = False

   // non-critical section
```

P_1

```
while (True):
   interested[1] = True
   turn = 0
   while (interested[0] and
   turn == 0):
        pass
   // critical section
   interested[1] = False

   // non-critical section
```

Peterson’s solution

+ Klarar i teorin samtliga tre krav.

- Slösar mycket tid på *busy-waiting* (spinning).
- Stödjer endast två processer (generaliserade varianter finns).
- Opålitlig med modern hårdvara och arkitektur, likt alla mjukvarubaserade lösningar.

Lås (icke-fungerande!)

```
is_locked = False
```

P_0

```
while (True):
   while (is_locked):
        pass
   is_locked = True
   // critical section
   is_locked = False

   // non-critical section
```

P_1

```
while (True):
   while (is_locked):
        pass
   is_locked = True
   // critical section
   is_locked = False

   // non-critical section
```

Observera att while (is_locked) motsvarar while (is_locked == True).

Lösningar som kräver hårdvarustöd

Kräver stöd för specifika atomära operationer i instruktionsuppsättningen:

- Mutex-lås.
- Semaforer.

Atomära operationer (atomic operation)

- Små operationer som är implementerade på hårdvarunivå, likt andra maskininstruktioner som ADD, STORE, etc.
- Går inte att avbryta (interrupt).
- För mutex-lås måste funktionen *acquire()* nyttja någon atomär operation.
- Vanliga exempel:
  - `TestAndSet()`
  - `CompareAndSwap()`

Ex. `TestAndSet()`

```
boolean testAndSet(boolean *lock) {
   boolean initial = *lock;
   *lock = true;

   return initial;
}
```

Förenklat:

- Låser låset.
- Returnerar vad låset var från början.

Mutex-lås (mutex locks)

```
locked = False
```

Funktioner som behövs

```
def acquire():
   while (testAndSet(locked)):
        pass

def release():
   locked = False
```

Hur funktionerna används i t.ex. Python (samma för alla processer)

```
lock = Lock()

while (True):
   lock.acquire()
   // critical section
   lock.release()
   // non-critical section
```

Mutex-lås (mutex locks)

+ Funkar för flera processer.
+ Funkar på moderna system.

- Slösar *potentiellt* mycket tid på *busy-waiting* (spinning).
- Kräver hårdvarustöd för atomära operationer.

Semaforer (semaphores)

```
A = amountOfResources
```

Funktioner som behövs

```
// som nedan fast med någon atomär
   operation

def wait():
   while (A <= 0):
        pass
   A = A - 1

def signal():
   A = A + 1
```

Hur funktionerna används i t.ex. Python (samma för alla processer)

```
sem = Semaphore(5)

while (True):
   sem.acquire() // or wait()
   // critical section
   sem.release() // or signal()
   // non-critical section
```

Semaforer (semaphores)

+ Funkar för flera processer.
+ Funkar på moderna system.
+ Kan användas för resurser som det finns flera instanser av.

- Slösar *potentiellt* mycket tid på *busy-waiting* (spinning). Kan använda någon form av kö för att undvika detta.
- Kräver hårdvarustöd för atomära operationer.

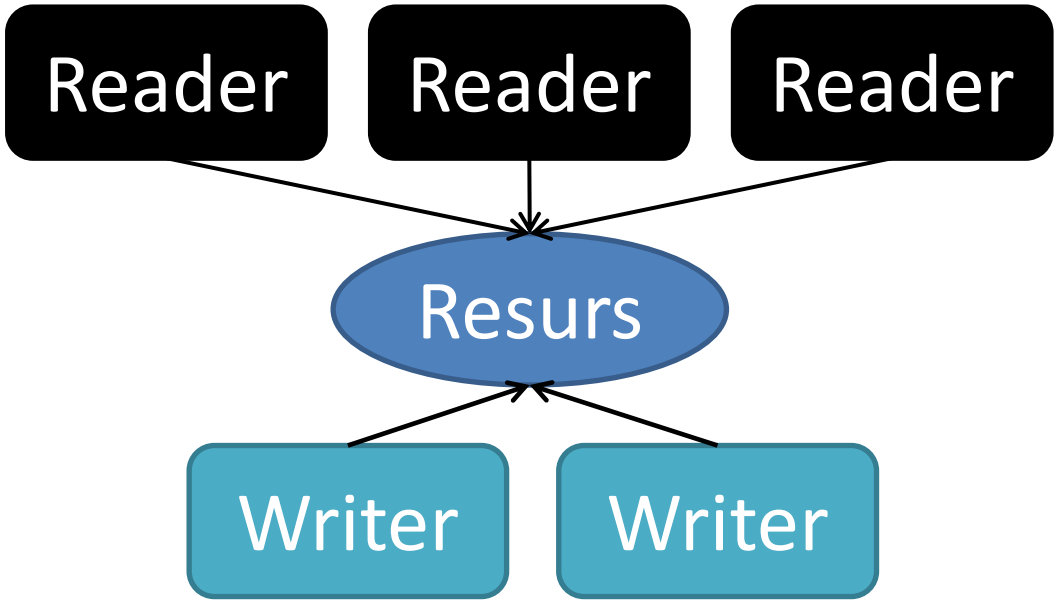
Readers-writers problem

Ett klassiskt synkroniseringsproblem

- Några processer läser, några skriver.

- Flera kan läsa samtidigt.
- Får ej skriva medan någon läser.

- Endast en kan skriva åt gången.
- Får ej läsa eller skriva medan någon annan skriver.



Readers-writers problem

- Problem 0 (grundproblemet):
  Mutual exclusion. Se till att endast en exekverande entitet kör i kritiska sektionen.

- Problem 1:
  Se till att ingen reader måste vänta, förutom om en writer redan har låst.

- Problem 2:
  När en writer är redo så ska den få skriva så snabbt som möjligt, d.v.s. inga nya readers får börja läsa.

Trådexempel i Python

- 100 trådar.
- Varje tråd har i uppgift att öka en global variabel med 1.
- Vad blir resultat med/utan synkronisering?

Synkronisering: Racetillstånd Sammanfattning

- **Racetillstånd** sker när processer utan kontroll försöker modifiera en gemensam resurs samtidigt.
- Den utsatta koddelen kallas för den **kritiska sektionen** (där gemensamma resurser hanteras).
- Det finns många olika lösningar på problemet. De bättre så som **mutex-lås** och **semaforer** kräver hårdvarustöd i form av specifika atomära operationer.